Differentiation on Spike Representation (DSR) and ImageNet Training Code Review

Author: Yechan Cho

Affiliation: Division of Bio-Medical Science & Technology

Course: Fundamentals of Deep Learning and PyTorch Programming

Date: 2025-12-14

Abstract

This report (i) summarizes the CVPR paper “Training High-Performance Low-Latency Spiking Neural Networks by Differentiation on Spike Representation (DSR)” and (ii) analyzes a provided PyTorch ImageNet training implementation (main.py + 4 supporting modules). The first two pages explain how spike trains can be converted into a continuous spike representation (rate / weighted-rate), enabling layer-wise learning through a sub-differentiable clamp mapping instead of BPTT. The last three pages dissect the training script: distributed execution, data pipeline, learning-rate schedule with warmup, training/validation loops, checkpointing, and optional per-layer threshold calibration / reassignment.

1. Introduction

Spiking Neural Networks (SNNs) communicate with discrete spikes and are attractive for neuromorphic hardware because computation can be event-driven and energy efficient. Training is difficult because spike generation uses a hard threshold (a discontinuous step), so naïve backpropagation produces unusable derivatives. Two popular families of solutions are: (1) surrogate-gradient training with Backpropagation Through Time (BPTT), and (2) ANN-to-SNN conversion, which often needs many time steps to match ANN activations. DSR proposes an alternative: differentiate on a continuous representation of spikes so that learning can proceed layer-by-layer without temporal backpropagation.

2. Differentiable Spiking Neural Networks via DSR

2.1 Key idea: representation-level learning

DSR encodes each layer’s output spike train into a continuous “spike representation” (SR). The forward pass still simulates spikes, but the backward pass uses gradients of a sub-differentiable mapping on SR tensors. This compresses temporal information into a single representation per layer and avoids BPTT-style time-unrolling.

Forward (SNN simulation) and **Backward (sub-differentiable mapping on SR)**

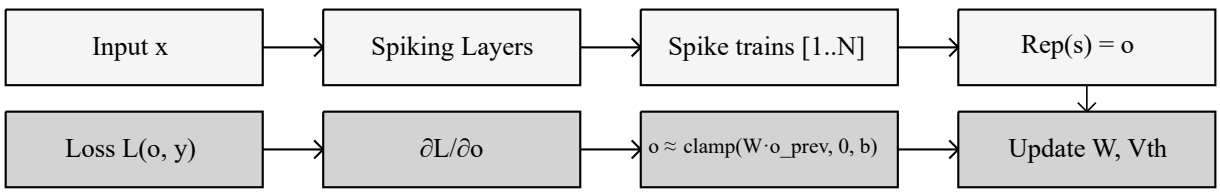

Figure 1. Conceptual DSR pipeline: spike simulation in forward; representation-level learning in backward.

2.2 Spike representation and clamp mapping (IF / LIF)

Let s = (s[1], …, s[N]) be a spike train over N time steps. DSR defines a representation r(s) via firing-rate coding (IF) or exponentially weighted firing-rate coding (LIF). With $\lambda = \exp(-\Delta t/\tau)$:

```
Spike representation r(s)
  IF  :  r(s) = (1/N) · Σ_{n=1..N} (V_th · s[n])
  LIF :  r(s) = V_th · Σ_{n=1..N} λ^{N-n} s[n] / ( Σ_{n=1..N} λ^{N-n} · Δt )

Layer-wise approximation on representations
o^i = r(s^i)  ≈  clamp( W^i · o^{i-1}, 0, b_i )
where b_i = V_th (IF) and b_i = V_th/Δt (LIF).
```

2.3 Reducing representation error (low latency)

The clamp form is derived under asymptotic arguments (large N). With small N (low latency), SR may deviate from the ideal mapping (“representation error”). The paper proposes two practical mitigations: (i) train the threshold V_th per layer while enforcing a positive lower bound, and (ii) introduce a firing-modification hyperparameter α so neurons fire when membrane potential exceeds α·V_th, improving gradient flow. The supplementary settings for ImageNet use Adam, cosine learning-rate annealing, 90 epochs, learning rate 0.001, and batch size 144—matching the defaults in the provided main.py.

2.4 Where these ideas appear in the provided code

Paper concept	Implementation (files / symbols)
Spike representation r(s)	rate_spikes(), weight_rate_spikes() (neuron.py)
Clamp-like gradients	IFFunction.backward / LIFFunction.backward (neuron.py)
Threshold training	Vth as nn.Parameter; lower bound via Vth_bound (neuron.py)
α firing modification	mem ≥ α·Vth in IF/LIF forward (neuron.py)
PreAct-ResNet backbone	preresnet_snn.py (spiking activations)
Optional Vth calibration	preresnet_cal_Vth.py + main.py --calculate_Vth/--load_Vth

3. main.py Training Code Review (ImageNet)

The training entry point is **main.py**. It extends the standard PyTorch ImageNet example with SNN-specific arguments (time steps, neuron thresholds, τ, Δt, α) and optional threshold calibration. It supports three execution modes: single GPU, DataParallel, and multi-process DistributedDataParallel (DDP).

3.1 High-level execution flow

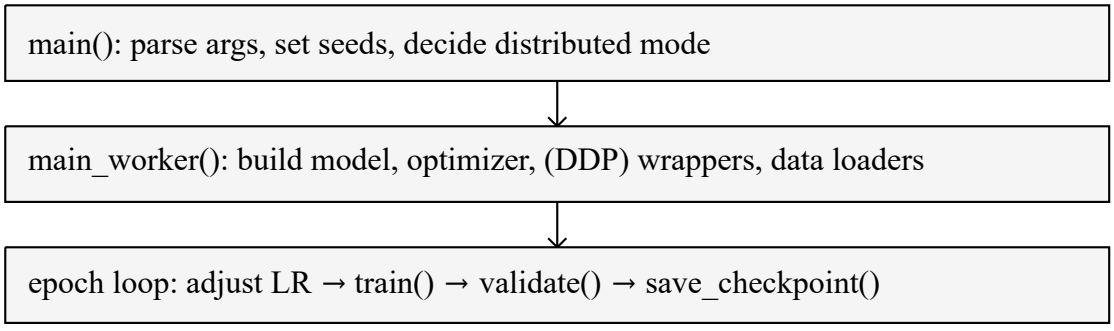

Figure 2. Call graph of main.py: main() selects the mode; main_worker() builds the training stack; the epoch loop trains, validates, and checkpoints.

Key modes controlled by flags:
• **--evaluate**: run validation only.
• **--resume**: restore model + optimizer to continue training.
• **--pre_train**: load weights (non-strict) for initialization.
• **--calculate_Vth**: run a few validation batches with a calibration network and save activation statistics.
• **--load_Vth**: load the saved statistics and overwrite per-layer thresholds.

3.2 SNN hyperparameters in main.py

main.py exposes DSR-style neuron controls. The most important arguments are: timesteps (N), Vth (initial threshold), tau (τ, for LIF), delta_t (Δt), alpha (α firing modification), train_Vth (whether thresholds are learnable), and Vth_bound (lower bound constraint). These are packed into a dictionary snn_setting and passed to the model constructor.

Listing 1: main.py (constructing snn_setting and loading calibrated thresholds)

```
141   # create model
142   snn_setting = {}
143   snn_setting['timesteps'] = args.timesteps
144   snn_setting['train_Vth'] = True if args.train_Vth == 1 else False
145   snn_setting['Vth'] = args.Vth
146   snn_setting['tau'] = args.tau
147   snn_setting['delta_t'] = args.delta_t
148   snn_setting['alpha'] = args.alpha
149   snn_setting['Vth_bound'] = args.Vth_bound
150   snn_setting['rate_stat'] = True if args.rate_stat == 1 else False
151
152   model = eval(args.arch + '(snn_setting, num_classes=1000)')
153
154   if args.load_Vth:
155       print('load threshold')
156       dict_Vth = torch.load(args.checkpoint + '/imagenet/' + args.load_Vth)
157       model = reassign_Vth(model, dict_Vth, 1.0)
```

3.3 Distributed training and device placement

When **--multiprocessing-distributed** is enabled, main() spawns one process per GPU and initializes a process group (NCCL by default). In this mode, batch size and worker count are divided by the number of GPUs per node, and the model is wrapped by DistributedDataParallel. Otherwise, the code falls back to DataParallel when multiple GPUs are visible.

3.4 Data pipeline (ImageNet)

Data loading follows conventional ImageNet preprocessing: RandomResizedCrop(224) + RandomHorizontalFlip for training, and Resize(256) + CenterCrop(224) for validation, followed by mean/std normalization. When distributed, a DistributedSampler ensures each process receives a distinct shard per epoch.

Listing 2: main.py (data transforms, optional Vth calibration, and the epoch loop — excerpt)

```
241   # Data loading code
242   traindir = os.path.join(args.data, 'train')
243   valdir = os.path.join(args.data, 'val')
244   normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
245                                    std=[0.229, 0.224, 0.225])
246
247   train_dataset = datasets.ImageFolder(
248       traindir,
249       transforms.Compose([
250           transforms.RandomResizedCrop(224),
251           transforms.RandomHorizontalFlip(),
252           transforms.ToTensor(),
253           normalize,
254       ]))
255
256
```

```
257    if args.distributed:
258        train_sampler = torch.utils.data.distributed.DistributedSampler(train_dataset)
259    else:
260        train_sampler = None
261
262    train_loader = torch.utils.data.DataLoader(
263        train_dataset, batch_size=args.batch_size, shuffle=(train_sampler is None),
264        num_workers=args.workers, pin_memory=True, sampler=train_sampler)
265
266    val_loader = torch.utils.data.DataLoader(
267        datasets.ImageFolder(valdir, transforms.Compose([
268            transforms.Resize(256),
269            transforms.CenterCrop(224),
270            transforms.ToTensor(),
271            normalize,
```

3.5 Optimization schedule and training loop

The optimizer is **Adam** by default (SGD is also supported). The learning rate follows cosine annealing once per epoch: lr = 0.5·lr0·(1 + cos(epoch/epochs·π)). Additionally, a linear warmup is applied for the first **--warmup** iterations of epoch 0. The train() loop is standard supervised learning: forward → cross-entropy → backward → optimizer.step, while reporting top-1/top-5 accuracy.

Listing 3: main.py (cosine learning-rate schedule and per-iteration warmup)

```
469    def adjust_learning_rate(optimizer, epoch, args):
470        lr = 0.5 * args.lr * (1 + math.cos(epoch/args.epochs*math.pi))
471        for param_group in optimizer.param_groups:
472            param_group['lr'] = lr
473
…
491    def adjust_warmup_lr(optimizer, citer, warmup, lr):
492        for param_group in optimizer.param_groups:
493            param_group['lr'] = lr * (citer + 1.) / warmup
```

3.6 Threshold calibration and reassignment (optional)

Besides learning V_th as parameters inside each spiking neuron, the script includes an optional calibration path. With **--calculate_Vth**, it runs a few validation batches using preresnet_cal_Vth (which records high-percentile activation maxima before ReLU) and saves a dictionary of thresholds. With **--load_Vth**, the thresholds are loaded and applied to every IFNeuron/LIFNeuron instance. For LIF neurons, the reassigned value is additionally scaled by Δt to match the representation bound b_i = V_th/Δt.

Listing 4: main.py (reassign_Vth — excerpt)

```
496    def reassign_Vth(net,dict_Vth, rate):
497        neuron_type = net.neuron_type
498        if neuron_type == 'if':
499            scale_factor = 1.0
500        elif neuron_type == 'lif':
501            scale_factor = net.snn_setting['delta_t']
502
503        net.relu1.Vth = nn.Parameter(torch.tensor(float(dict_Vth['storein1'].item() *rate *
       scale_factor)))
504        net.relu2.Vth = nn.Parameter(torch.tensor(float(dict_Vth['storein2'].item() *rate *
       scale_factor)))
505        net.relu3.Vth = nn.Parameter(torch.tensor(float(dict_Vth['storein3'].item() *rate *
       scale_factor)))
506        net.relu4.Vth = nn.Parameter(torch.tensor(float(dict_Vth['storein4'].item() *rate *
       scale_factor)))
507
508    for j in ['1', '2', '3', '4']:
509        for i in range(eval('lens' + j)):
510            exec('net.layer' + j + '[' + str(i) + '].relu1.Vth' + '''=
       nn.Parameter(torch.tensor(float(dict_Vth['layer' + j + '[' + str(i) + ']' +
       '.storein1'][0].item() *rate * scale_factor)))''')
```

```
511           exec('net.layer' + j + '[' + str(i) + '].relu2.Vth' + '''=
      nn.Parameter(torch.tensor(float(dict_Vth['layer' + j + '[' + str(i) + ']' +
      '.storein2'][0].item() *rate * scale_factor)))''')
512           if 'conv3' in dir(net.layer1[0]):
513               exec('net.layer' + j + '[' + str(i) + '].relu3.Vth' + '''=
      nn.Parameter(torch.tensor(float(dict_Vth['layer' + j + '[' + str(i) + ']' +
      '.storein3'][0].item() *rate * scale_factor)))''')
514   return net
```

4. Conclusion

DSR makes SNN training practical at low latency by learning on continuous spike representations and treating each layer as a sub-differentiable clamp mapping. The provided codebase closely mirrors this idea: neuron.py implements IF/LIF neurons with representation-level gradients (including α firing modification and threshold constraints), preresnet_snn.py provides a PreAct-ResNet backbone with spiking activations, and main.py organizes ImageNet training with cosine LR + warmup, DDP support, checkpointing, and optional threshold calibration/reassignment.

References

[1] Qingyan Meng et al. “Training High-Performance Low-Latency Spiking Neural Networks by Differentiationon Spike Representation.” arXiv:2205.00459v2, 2023.
[2] PyTorch ImageNet Example (torchvision training template for DataParallel / DistributedDataParallel).